

Employee Management REST API

Overview

The **Employee Management REST API** is a Spring Boot application developed using **Spring Boot, Spring Data JPA, Hibernate, and Oracle/MySQL Database**. It provides a set of RESTful web services for performing **CRUD (Create, Read, and Delete)** operations on employee records.

The API demonstrates the implementation of the **Controller-Service-Repository architecture**, enabling clean separation of concerns and maintainable code. It exposes HTTP endpoints that can be tested using **Postman** or any REST client.

This collection is designed for students, developers, and learners to understand how REST APIs are developed using Spring Boot and how data is persisted using Spring Data JPA.

Features

- RESTful API design
 - Create Employee
 - Retrieve all Employees
 - Retrieve Employee by ID
 - Delete Employee by ID
 - Delete all Employees
 - Health Check Endpoint
 - JSON Request and Response
 - Spring Data JPA Integration
 - Hibernate ORM
 - Oracle/MySQL Database Support
 - Easy testing with Postman
-

Technology Stack

Technology	Description
Java 17	Programming Language

Technology	Description
Spring Boot	Backend Framework
Spring MVC	REST API Development
Spring Data JPA	Data Access Layer
Hibernate	ORM Framework
Oracle / MySQL	Relational Database
Maven	Dependency Management
Postman	API Testing
Embedded Tomcat	Web Server

Base URL

`http://localhost:8080`

If HTTPS is configured:

`https://localhost:8443`

Available Endpoints

Method	Endpoint	Description
GET	<code>/check</code>	Verify API availability
POST	<code>/employee</code>	Create a new employee
GET	<code>/findall</code>	Retrieve all employees
GET	<code>/findbyid/{id}</code>	Retrieve employee by ID
DELETE	<code>/deletebyid/{id}</code>	Delete employee by ID
DELETE	<code>/delete</code>	Delete all employee records

API Details

1. Health Check

GET `/check`

Returns a simple message indicating that the API is running successfully.

Response

This is https text

2. Add Employee

POST /employee

Creates a new employee record.

Request Body

```
{
  "ename": "Rahul Sharma",
  "edept": "Information Technology"
}
```

Success Response

```
{
  "eid": 1,
  "ename": "Rahul Sharma",
  "edept": "Information Technology"
}
```

3. Retrieve All Employees

GET /findall

Returns all employee records stored in the database.

Response

```
[
  {
    "eid":1,
    "ename":"Rahul",
    "edept":"IT"
  },
  {
    "eid":2,
    "ename":"Mohan",
    "edept":"HR"
  }
]
```

4. Retrieve Employee by ID

GET /findbyid/{id}

Example

GET /findbyid/1

Response

```
{
  "eid":1,
  "ename":"Rahul",
  "edept":"IT"
}
```

5. Delete Employee by ID

DELETE /deletebyid/{id}

Example

DELETE /deletebyid/1

Deletes the specified employee record from the database.

6. Delete All Employees

DELETE /delete

Deletes every employee record from the database.

Request Headers

For POST requests

Content-Type: application/json

If security is enabled

Authorization: Bearer <JWT Token>

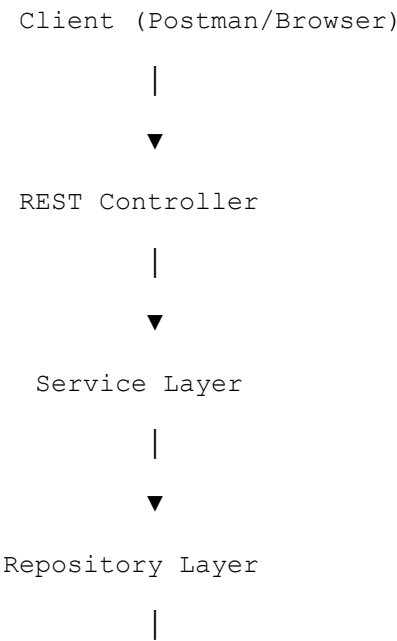
Sample Employee Object

```
{
  "eid": 1,
  "ename": "Rahul Sharma",
  "edept": "Computer Science"
}
```

HTTP Status Codes

Status Code	Description
200 OK	Request processed successfully
201 Created	Resource created successfully
400 Bad Request	Invalid request data
401 Unauthorized	Authentication required
404 Not Found	Employee not found
500 Internal Server Error	Server-side exception

Project Architecture



▼
Spring Data JPA + Hibernate

|

▼
Oracle / MySQL Database

Project Highlights

- Built using **Spring Boot 3**
 - Implements **RESTful Web Services**
 - Uses **Spring Data JPA** for database operations
 - Implements **Dependency Injection**
 - Uses **Hibernate ORM**
 - Supports **Oracle** and **MySQL**
 - Easily testable with **Postman**
 - Layered architecture for scalability and maintainability
-

How to Run

1. Clone the project.
 2. Configure the database connection in `application.properties`.
 3. Create the required database schema.
 4. Run the Spring Boot application.
 5. Open Postman.
 6. Import the API collection.
 7. Test the available endpoints.
-

Documentation

Interactive API documentation is available at:

[Employee API Documentation](#)
